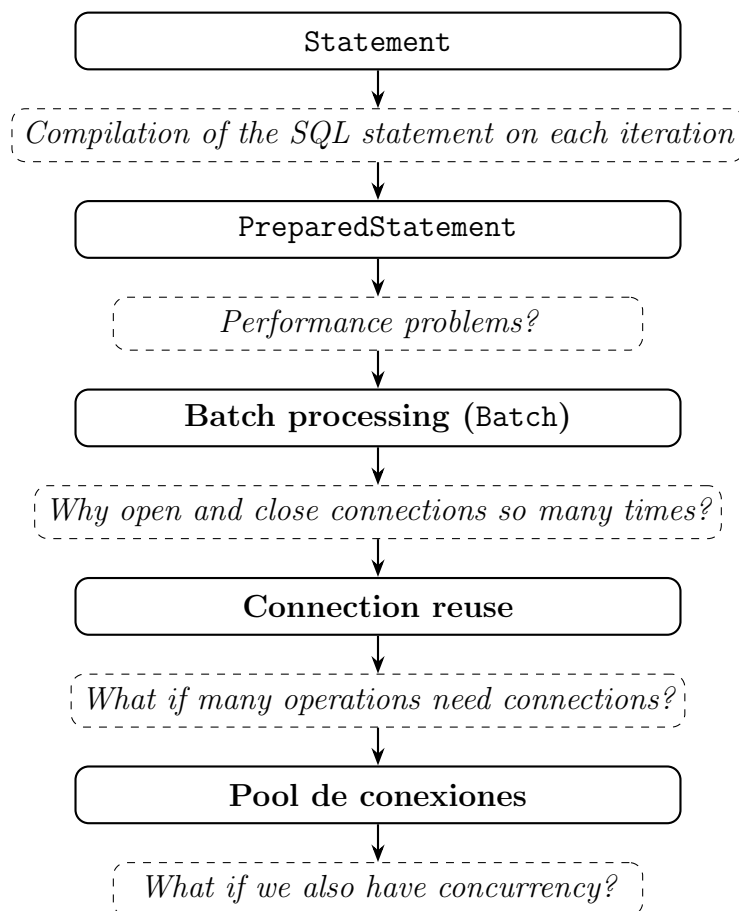


Practical Assignment 1: Database Access Costs

1 Objectives

- Connections to various DBMSs: Oracle, HSQLDB.
- Comparison of `Statement`/`PreparedStatement`.
- Use of a connection pool and comparison of execution times.

The sequence of the assignment will be as follows:



2 Preparation

1. Download the material from the Virtual Campus (HSQLDB database, Oracle script, drivers).
2. Unzip the `cws_database` file and start the HSQLDB server (run `1.startup`).
3. Connect to Oracle (e.g. SQLDeveloper). The connection details are in section 3.
4. Create a workspace directory and start Eclipse.

2.1 Build the Oracle database

Once connected to Oracle, it is necessary to delete the database tables and create the new ones.

- a. If you have taken the course in previous years, you must first delete the tables from previous years from the database. To do so, execute the following command:

```
SELECT 'DROP TABLE ' || table_name || ' CASCADE CONSTRAINTS; '
FROM user_tables
WHERE table_name LIKE 'T%'
```

- b. The result is the list of deletion commands that you must execute. Copy it and run it.
- c. Run the script `script_create_carworkshop_Oracle.sql` to create the new tables.

2.2 Create and configure an Eclipse project

- Create an Eclipse project to write the exercises.
- Configure the project
 - Create a folder (not source code) `lib` in the project. Copy the `.jar` files for the two drivers (Oracle, HSQLDB) there.
 - Configure the application's build path to reference both of them (Add external JAR).

3 Connection details

Parameter	Oracle	HSQLDB
DRIVER	thin	hsqldb
HOST / IP	156.35.94.98	localhost
PUERTO	1521	–
SID (id)	desa19	–
USER	U0	sa
PASS	password Oracle	""

4 Exercises

4.1 Compare the performance of Statement vs. PreparedStatement

Create a class called `StatementBenchmark` for the first exercise.

The program must connect to the database and query some data. The objective is to measure the time taken by a task, carrying it out in two ways: using *Statement*, and *PreparedStatement*.

To make the difference in execution time significant, the statement must be executed enough times, for example **1000, changing slightly** each time. In the HSQLDB database it can even be run 10000 times; in Oracle this would take a long time to execute, while waiting for the output.

The following statement is suggested: **Retrieve from the TINVOICES table the identifier of an invoice whose amount due is less than the value of the loop control variable.**

To measure execution times you can use `System.currentTimeMillis()`, which returns the date/-time in milliseconds since 01/01/1970.

4.1.1 Is using PreparedStatement more efficient than Statement?

To check this, run each version several times and record the results.

Execution times

Time (ms)	HSQLDB		ORACLE	
	Statement	PreparedStatement	Statement	PreparedStatement
run 1				
run 2				
run 3				

Is there a significant difference between Statement and PreparedStatement in HSQLDB? And in Oracle? What factors could explain the results?

4.2 Batch processing

In the previous exercise, we executed the operations one at a time.

When the database is located on a remote server (as is the case with Oracle), each operation may involve communication between the client program and the database server.

What happens when we have to execute many operations? Can we reduce the cost of these communications by grouping the operations into a batch?

JDBC provides the mechanism of **batch processing**, which allows grouping several operations into a batch, reducing the number of interactions with the server.

Thus, if a significant part of the operation cost is due to the communication with the server, grouping several operations into a batch should significantly reduce execution time.

The purpose of this exercise is to measure and compare the impact of executing several SQL statements in a batch versus executing them one at a time.

Create a class called **BatchBenchmark**. It must connect to the Oracle database and update (queries cannot be executed in a batch) varias filas utilizando dos estrategias:

- executing the operations individually;
- grouping them using *JDBC batch processing*. You can find an example here: <https://www.baeldung.com/jdbc-batch-processing>

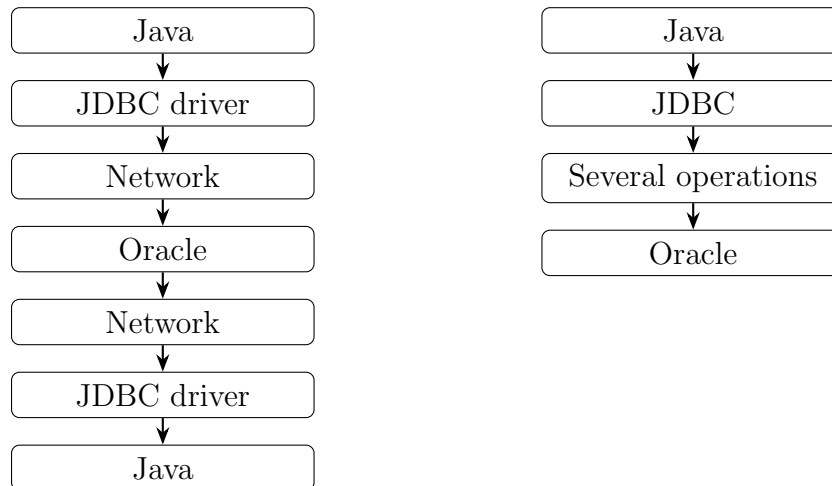
The following statement is suggested: **Update the TINVOICES table by adding the value of the loop control variable to the amount field when the status is PAID.**

1. One-by-one execution

- Create the connection
- Create the prepared statement
- Run a loop. On each iteration:
 - Complete the statement to be executed.
 - Execute it.

2. Batch execution

- Create the connection
- Create the prepared statement
- Run a loop. On each iteration:
 - Complete the statement to be executed.
 - Add it to the batch (`addBatch`).
- Execute the batch (`executeBatch`).



To check the possible improvement from batch execution, run each scenario several times and record the results.

Execution times

Time (ms)	ORACLE	
	without batching	with batching
run 1		
run 2		
run 3		

It is worth reading a very interesting discussion about the topics covered in this exercise here.

4.3 Check the cost of opening and closing connections

A database connection is an expensive resource, both in terms of time and memory, since it manages all communication between the client application and the server.

Create a class called `ConnectionBenchmark` for the second exercise.

The purpose of this exercise is to measure and compare the impact of opening and closing connections versus reusing them, by developing a program that evaluates execution times in two different scenarios:

The statement to be executed will be the same as in the previous exercise, and we will use `PreparedStatement` in both cases.

1. Connection inside the loop

- Run the loop. On each iteration:
 - Create a connection.

- Complete the statement.
- Execute the statement.
- Close the connection.

2. Connection outside the loop

- Create the connection before starting the loop.
- Run the loop. On each iteration:
 - Complete the statement.
 - Execute the statement.
- Close the connection after the loop finishes.

Comparing both scenarios will show the cost difference between repeatedly establishing a connection and reusing an open connection for several operations.

To check this, run each scenario several times and record the results.

Execution times

Tiempo (ms)	HSQLDB		ORACLE	
	in loop	out loop	in loop	out loop
run 1				
run 2				
run 3				

4.4 Compare the cost of establishing a physical connection to the DBMS versus reusing an existing connection from a connection pool

The objective of this exercise is **to compare the cost of creating physical connections to the database with using a connection pool for it.**

Utilizaremos el pool de conexiones c3p0, cuyos drivers se encuentran en la carpeta lib (descargados desde <https://www.mchange.com/projects/c3p0/>).

Create a class called `ConnectionPoolBenchmark` for this exercise.

In it, the database will be accessed several times and an operation will be performed each time. As in the other exercises, two different scenarios will be implemented in which, for each database operation:

- **Without a connection pool:** its own connection is created and closed.
- **With a connection pool:** a logical connection is obtained from the pool and returned to the pool when finished.

The purpose is **to measure execution times** in both cases and analyze how using a connection pool improves the efficiency and performance of database access.

Example of a loop without a pool

```
for (int i = 0; i < 1000; i++) {
    crearConexion();
    ejecutar sentencia();
    cerrarConexion();
}
```

Example of a loop with a pool

```
crearPoolConexiones();
for (int i = 0; i < 1000; i++) {
    getConexionFromPool();
    ejecutar sentencia();
    releaseConexion();
}
```

In the second scenario, the creation and configuration of a connection pool with the following characteristics (library default values) is added to the connection and database access instructions:

- Maximum number of connections (`maxPoolSize`): 15.
- Minimum number of connections (`minPoolSize`): 3.
- Initial pool size (`initialPoolSize`): 3.

4.4.1 Example of c3p0 initialization

For reference, the following code excerpt available on the c3p0 website can be used to create a `PooledDataSource`:

```
DataSource ds_unpooled = DataSources.unpooledDataSource(URL, USER, PASS);

Map<String, Object> overrides = new HashMap<>();

/* The following three instructions can be omitted
   if the values being used are the default values */
overrides.put("minPoolSize", 3);
overrides.put("maxPoolSize", 15);
overrides.put("initialPoolSize", 3);

DataSource ds_pooled =
    DataSources.pooledDataSource(ds_unpooled, overrides);

// To obtain a connection from the pool
Connection con = ds_pooled.getConnection();
```

4.4.2 Repeated execution of each scenario

Run each scenario several times and record the following.

Execution times

	HSQldb	ORACLE
Without a connection pool		
With a connection pool (Measurement A)		
With a connection pool (Measurement B)		

Measurement A — Including pool creation Measure the times **including pool creation** of the connection pool. This helps answer the question: Is it worthwhile to use a pool for a small execution?

Measurement A — Pool already created Measure the time **after creating the pool**, just before starting the loop. This answers the question: What is the cost of using a pool once it has already been initialized?

If the task were sufficiently costly, it would be even better to divide it into subtasks (Threads), each of which would execute part of them.